

Geração de Certificados Digitais Simulados — ICP-Brasil

Documento de descrição do passo a passo + fundamentação teórica Atividade educacional — simulação do processo de geração de um certificado digital X.509 em conformidade com a estrutura da Infraestrutura de Chaves Públicas Brasileira (ICP-Brasil).

⚠ **Aviso importante:** O certificado gerado nesta atividade é **autoassinado** e **simulado**, destinado exclusivamente a fins **educacionais**. Ele **não possui validade jurídica** e **não substitui** um certificado emitido por uma Autoridade Certificadora (AC) credenciada na ICP-Brasil.

Sumário

Parte I — Prática 1. Visão geral 2. Tecnologias utilizadas 3. Estrutura de arquivos do projeto 4. Pré-requisitos 5. Passo a passo — interface web (Docker) 6. Passo a passo — modo linha de comando (CLI) 7. Inspeccionando o certificado gerado 8. Detalhamento técnico do código 9. Arquitetura da aplicação web

Parte II — Fundamentação teórica (nível Ciência da Computação) 10. Criptografia simétrica × assimétrica e modelo híbrido 11. Fundamentos matemáticos do RSA 12. Padding e esquemas de cifragem/assinatura 13. Funções de hash criptográficas 14. Assinatura digital 15. O padrão X.509 e o Distinguished Name 16. Codificação: ASN.1, DER, Base64 e PEM 17. Extensões X.509 v3 18. Validação da cadeia de confiança 19. Revogação: CRL e OCSP 20. TLS/HTTPS — onde os certificados são usados 21. Criptografia de curvas elípticas (ECC) 22. Ameaça quântica e criptografia pós-quântica 23. A família de padrões PKCS 24. ICP-Brasil em profundidade 25. Boas práticas de segurança 26. Referências 27. Glossário

Parte I — Prática

1. Visão geral

A atividade reproduz, de forma simplificada, as três etapas centrais da emissão de um certificado digital:

1. **Geração de um par de chaves** criptográficas RSA (chave privada + chave pública).
2. **Criação de um certificado X.509** contendo os dados do titular (DN — *Distinguished Name*).
3. **Armazenamento seguro** da chave privada e do certificado em arquivos `.pem` separados.

O projeto oferece **duas formas de uso**:

- **Interface web** (recomendada): formulário visual, listagem dos certificados gerados, modal com todos os detalhes, certificado decodificado (estilo OpenSSL) e download dos arquivos. Inclui ainda uma aba **didática** com toda a fundamentação teórica.
- **Linha de comando (CLI)**: gera os arquivos diretamente na pasta `saida/`.

Todo o processo é executado dentro de um **contêiner Docker**, garantindo um ambiente isolado e reproduzível, sem necessidade de instalar Python ou bibliotecas na máquina local.

2. Tecnologias utilizadas

Componente	Finalidade
Python 3.12	Linguagem de programação
<code>cryptography</code>	Biblioteca para geração de chaves e certificados X.509
Flask	Microframework web que serve a interface e a API
Docker	Empacotamento e execução em ambiente isolado
Docker Compose	Orquestração simplificada da execução
HTML/CSS/JS	Interface (abas, formulário, listagem, modal, ícones SVG)

3. Estrutura de arquivos do projeto

```
SS/
├── gerar_certificado.py # Lógica de geração (chaves + certificado X.509) – também usável via CLI
├── app.py               # Servidor Flask: interface web + API (gerar, histórico, modal, download)
├── templates/
│   └── index.html      # Interface (abas Gerar/Explicação, listagem, modal, conteúdo didático)
├── requirements.txt     # Dependências Python (cryptography, Flask)
├── Dockerfile           # Receita de construção da imagem Docker
├── docker-compose.yml   # Orquestração (serviço web + serviço cli opcional)
├── .dockerignore        # Arquivos ignorados na construção da imagem
├── PASSO_A_PASSO.md     # Este documento
├── README.md            # Instruções rápidas de uso
├── saida/
│   ├── private_key.pem # (criada na execução) recebe os arquivos .pem
│   └── certificate.pem
```

4. Pré-requisitos

- **Docker** instalado e em execução (Docker Desktop no Windows/Mac, ou Docker Engine no Linux).
- Verifique a instalação com: `bash docker --version docker compose version`

5. Passo a passo — interface web (Docker)

Passo 1 — Abrir o terminal na pasta do projeto

No Windows (PowerShell):

```
cd C:\Users\word2\Downloads\SS
```

Passo 2 — Construir a imagem e subir o serviço web

```
docker compose up -d --build web
```

- `--build` reconstrói a imagem (instala Python, `cryptography` e Flask).
- `-d` executa em segundo plano (*detached*).

Passo 3 — Abrir a aplicação no navegador

Acesse: **`http://localhost:5000`**

Passo 4 — Gerar um certificado

Na aba **Gerar Certificado**, preencha:

Campo	Significado	Exemplo
País (2 letras)	Código do país (ISO 3166-1 alfa-2)	BR
Estado/Província	Unidade federativa	Goiás
Cidade	Localidade	Goiania
Organização	Nome da organização (simulada)	Universidade XYZ
Nome Comum (CN)	Identificação do titular	Aluna de Teste
E-mail	Vai na extensão SubjectAlternativeName	aluna@exemplo.com.br
Tamanho da chave	2048 / 3072 / 4096 bits	2048
Validade (dias)	365 = 1 ano	365
Senha da chave	(opcional) protege a chave privada	senhaForte

Clique em **Gerar Certificado**. O certificado é criado, **adicionado à listagem** e o **modal** com todos os detalhes abre automaticamente.

Passo 5 — Consultar a listagem e o modal

- A tabela **Certificados gerados** lista todos os certificados da sessão (CN, organização, local, tamanho da chave, validade e data de criação).
- **Clique em uma linha** (ou no botão *Detalhes*) para abrir o **modal**, que mostra:
- Tabela de informações completas (número de série, algoritmo, validade...);
- Card de **arquivos gerados** com botões de download;
- **Certificado decodificado** (estilo OpenSSL), incluindo extensões e *fingerprints*;
- O conteúdo bruto em **PEM** do certificado e da chave (com botão *Copiar*).
- Fecha o modal clicando fora, no **X** ou pressionando **ESC**.

Passo 6 — Localizar os arquivos em disco

Os arquivos do último certificado gerado também são gravados na pasta local:

```
saida/certificate.pem    → certificado digital (público)
saida/private_key.pem    → chave privada (GUARDAR COM SEGURANÇA)
```

Passo 7 — Parar o serviço (quando terminar)

```
docker compose down
```

6. Passo a passo — modo linha de comando (CLI)

Caso prefira gerar os arquivos sem interface gráfica, há um serviço `cli` no `docker-compose.yml`:

```
docker compose run --rm cli
```

Saída esperada:

```
=====
GERAÇÃO DE CERTIFICADO DIGITAL SIMULADO - ICP-Brasil (educacional)
=====
[OK] Par de chaves RSA de 2048 bits gerado com sucesso.
[OK] Certificado X.509 autoassinado criado e assinado (SHA-256).
      Início da validade : ...
      Término da validade: ...
[OK] Chave privada salva em 'private_key.pem' ...
[OK] Certificado salvo em 'certificate.pem'.
=====
```

Personalizando via parâmetros

```
docker compose run --rm cli \
  --bits 4096 \
  --dias 365 \
  --pais BR \
  --estado "Rio de Janeiro" \
  --cidade "Niteroi" \
  --organizacao "Minha Empresa Educacional" \
  --nome-comum "Joao da Silva" \
  --email "joao.silva@exemplo.com.br" \
  --senha-chave "minhaSenhaForte"
```

Parâmetros disponíveis

Parâmetro	Padrão	Descrição
<code>--bits</code>	2048	Tamanho da chave RSA (mín. recomendado 2048)
<code>--dias</code>	365	Dias de validade (365 = 1 ano)
<code>--pais</code>	BR	País
<code>--estado</code>	Sao Paulo	Estado/Província
<code>--cidade</code>	Sao Paulo	Localidade
<code>--organizacao</code>	Organizacao Simulada LTDA	Nome da organização
<code>--nome-comum</code>	Usuario de Teste	Nome comum (CN)
<code>--email</code>	usuario.teste@exemplo.com.br	E-mail (SubjectAlternativeName)
<code>--senha-chave</code>	(vazio)	Protege a chave privada com senha
<code>--saida-chave</code>	private_key.pem	Nome do arquivo da chave privada
<code>--saida-cert</code>	certificate.pem	Nome do arquivo do certificado

7. Inspeccionando o certificado gerado

A interface já mostra o certificado decodificado. Para conferir também por fora, com o **OpenSSL**:

```
# Texto completo do certificado
openssl x509 -in saida/certificate.pem -text -noout

# Apenas titular, emissor e validade
openssl x509 -in saida/certificate.pem -noout -subject -issuer -dates

# Impressão digital (fingerprint) SHA-256
openssl x509 -in saida/certificate.pem -noout -fingerprint -sha256
```

Para inspecionar a chave privada (se protegida, pedirá a senha):

```
openssl pkey -in saida/private_key.pem -text -noout
```

8. Detalhamento técnico do código

O arquivo `gerar_certificado.py` implementa as três tarefas:

Tarefa 1 — Geração do par de chaves RSA

Função `gerar_par_de_chaves(tamanho_bits)`:

```
rsa.generate_private_key(public_exponent=65537, key_size=tamanho_bits)
```

- Algoritmo **RSA** com expoente público 65537 (primo de Fermat $2^{16}+1$, padrão seguro).
- Tamanho configurável (padrão **2048 bits**); avisa se < 2048 .
- A chave pública é derivada da privada (`chave_privada.public_key()`).

Tarefa 2 — Criação do certificado X.509

Função `criar_certificado(chave_privada, dados, dias)`: - Monta o **DN (Distinguished Name)**: `C, ST, L, O, CN`. - Como é **autoassinado**, `subject` (titular) = `issuer` (emissor). - **Validade**: `not_valid_before` = agora; `not_valid_after` = agora + dias. - Adiciona **extensões**: `BasicConstraints(ca=False)`, `KeyUsage` e `SubjectAlternativeName`. - **Assina** o certificado com a chave privada usando **SHA-256**.

Tarefa 3 — Armazenamento seguro

Funções `salvar_chave_privada()` e `salvar_certificado()`: - **Chave privada**: formato **PKCS#8** em PEM, opcionalmente cifrada por senha (`BestAvailableEncryption`). - **Certificado**: formato **PEM** padrão. - Arquivos salvos **separadamente**.

9. Arquitetura da aplicação web

O arquivo `app.py` é um servidor **Flask** que reutiliza a lógica de `gerar_certificado.py` e expõe uma **API**:

Rota	Método	Função
/	GET	Serve a interface (<code>index.html</code>)
/gerar	POST	Gera o certificado, salva em disco e adiciona ao histórico
/historico	GET	Lista resumida dos certificados gerados na sessão
/certificado/<id>	GET	Detalhes completos de um certificado (alimenta o modal)
/download/<id>/<tipo>	GET	Baixa o certificado/chave de um item específico
/download/<tipo>	GET	Baixa o último certificado/chave gerados (compatibilidade)

- O **histórico** fica em memória do servidor (some ao reiniciar o contêiner) — adequado a uma atividade educacional.
- Cada certificado é identificado pelo seu **número de série em hexadecimal**.
- O endpoint `/gerar` também produz o **texto decodificado** do certificado (função `_texto_certificado`), no estilo `openssl x509 -text`, com extensões e *fingerprints* SHA-256/SHA-1.
- CORS está habilitado para permitir o uso a partir do painel de pré-visualização.

Fluxo de uma geração (visão de alto nível):

```

Navegador (form)
  | POST /gerar (JSON com os dados)
  v
Flask /gerar → gerar_par_de_chaves() → criar_certificado() → serializa PEM
  |                                     |
  |                                     |→ salva em saida/*.pem
  |                                     |→ adiciona ao histórico (memória)
  v
Resposta { ok, id }
  |
  v
Navegador: recarrega /historico (tabela) e abre /certificado/<id> (modal)

```

Parte II — Fundamentação teórica (nível Ciência da Computação)

10. Criptografia simétrica × assimétrica e modelo híbrido

	Simétrica	Assimétrica
Chaves	Uma única chave secreta compartilhada	Par: pública + privada
Velocidade	Muito rápida	Lenta (números grandes)
Problema	Como combinar a chave em segredo?	Resolve a distribuição de chaves
Exemplos	AES, ChaCha20, 3DES	RSA, ECDSA, Diffie-Hellman

Como a criptografia assimétrica é lenta, na prática usa-se um **esquema híbrido**: a chave pública (RSA/ECC) transporta uma **chave de sessão simétrica** aleatória, e todo o tráfego real é cifrado com algoritmo simétrico rápido (ex.: AES). É exatamente o que ocorre no TLS/HTTPS (seção 20).

11. Fundamentos matemáticos do RSA

O RSA (Rivest-Shamir-Adleman, 1977) baseia-se na **aritmética modular** e na dificuldade de **fatorar** o produto de dois primos grandes.

11.1 Geração das chaves

```

Escolhe dois primos grandes p, q (≈1024 bits cada para n de 2048 bits)
n = p × q                                # módulo (presente nas duas chaves)
φ(n) = (p - 1) × (q - 1)                 # função totiente de Euler
Escolhe e tal que 1 < e < φ(n) e mdc(e, φ(n)) = 1    # normalmente e = 65537
d ≡ e-1 (mod φ(n))                       # inverso multiplicativo (Euclides estendido)

Chave pública = (n, e)
Chave privada = (n, d)                   # p, q e φ(n) são mantidos em segredo/descartados

```

11.2 Cifragem e decifragem

```

cifrar:    c = me mod n                    # m = mensagem como número < n
decifrar:  m = cd mod n                    # recupera a mensagem original

```

Funciona porque, pelo teorema de Euler, $m^{(e \cdot d)} \equiv m \pmod{n}$. A assinatura é a operação espelhada: assina-se com **d** (privada) e verifica-se com **e** (pública).

11.3 Exemplo numérico (brinquedo — inseguro, apenas didático)

```
p = 61, q = 53
n = 61 × 53 = 3233
φ(n) = 60 × 52 = 3120
e = 17 (mdc(17, 3120) = 1)
d = 2753 (pois 17 × 2753 = 46801 ≡ 1 mod 3120)
```

Mensagem m = 65

Cifra: c = 65¹⁷ mod 3233 = 2790

Decifra: m = 2790²⁷⁵³ mod 3233 = 65 ✓

11.4 Por que é seguro

A chave pública revela n e e . Para obter d seria preciso conhecer $\phi(n)$, o que exige **fatorar n** em $p \times q$. Para n de 2048 bits, não há algoritmo clássico viável — daí o tamanho mínimo recomendado. O expoente $65537 = 2^{16} + 1$ é primo e tem poucos bits 1, acelerando a exponenciação sem enfraquecer a segurança.

12. Padding e esquemas de cifragem/assinatura

RSA "puro" (*textbook*) é **inseguro**: é determinístico e maleável. Por isso aplica-se sempre um **preenchimento (padding)** antes da operação:

Esquema	Uso	Observação
PKCS#1 v1.5	Cifragem e assinatura (legado)	Assinatura aceitável; cifragem desencorajada
OAEP	Cifragem	Padding probabilístico moderno para cifrar
PSS	Assinatura	Padding probabilístico moderno para assinar (recomendado)

Nesta atividade, a biblioteca `cryptography` aplica o padding adequado ao assinar o certificado (PKCS#1 v1.5 com SHA-256, padrão do `CertificateBuilder`).

13. Funções de hash criptográficas

Uma função de hash $H(x)$ transforma uma entrada de qualquer tamanho em um **resumo de tamanho fixo** (digest). Propriedades exigidas:

- **Resistência à pré-imagem:** dado h , é inviável achar x tal que $H(x) = h$.
- **Resistência à segunda pré-imagem:** dado x , é inviável achar $x' \neq x$ com $H(x') = H(x)$.
- **Resistência a colisão:** é inviável achar *qualquer* par $x \neq x'$ com o mesmo hash.

O **efeito avalanche** garante que mudar 1 bit da entrada altera ~50% dos bits do digest. A família **SHA-2** (SHA-256/384/512) é o padrão atual; **MD5** e **SHA-1** estão **quebrados** (colisões práticas) e não devem ser usados em assinatura. SHA-256 produz 256 bits (32 bytes).

```
SHA-256("certificado") → e3b0c44298fc1c149afbf4c8996fb924... (64 hex / 256 bits)
SHA-256("certificado") → 7d865e959b2466918c9863... (1 letra trocada → totalmente diferente)
```

14. Assinatura digital

A assinatura combina **hash** + **criptografia assimétrica**, provando *autoria* e *integridade*.

Ao assinar (emissor): 1. Calcula o **hash** do documento (ex.: SHA-256). 2. **Cifra o hash com a chave privada** → resultado é a **assinatura**. 3. Envia documento + assinatura + certificado (que contém a chave pública).

Ao verificar (destinatário): 1. Recalcula o **hash** do documento recebido. 2. **Decifra a assinatura com a chave pública**, obtendo o hash original. 3. Se os hashes forem **iguais** → assinatura válida (não houve alteração e a autoria está provada).

Por que assinar o hash e não o documento todo? Operações RSA atuam sobre blocos do tamanho da chave; assinar um hash de tamanho fixo é eficiente e, pela resistência a colisão, tão seguro quanto.

Fingerprint do certificado: é o hash (ex.: SHA-256) do certificado inteiro, usado para identificá-lo e compará-lo de forma única.

15. O padrão X.509 e o Distinguished Name

X.509 (ITU-T; perfil de Internet na RFC 5280) padroniza a estrutura do certificado.

Campo	Descrição
Subject (Titular)	Quem o certificado identifica (descrito pelo DN)
Issuer (Emissor)	Quem emitiu e assinou
Public Key	Chave pública do titular
Serial Number	Número de série único atribuído pelo emissor
Validity	<code>notBefore</code> e <code>notAfter</code>
Signature	Assinatura digital do emissor (aqui SHA-256 com RSA)
Extensions	KeyUsage, BasicConstraints, SubjectAlternativeName, etc.

Distinguished Name (DN):

Sigla	Atributo	Exemplo
<code>C</code>	Country (País)	<code>BR</code>
<code>ST</code>	State (Estado)	<code>Goiás</code>
<code>L</code>	Locality (Cidade)	<code>Goiania</code>
<code>O</code>	Organization	<code>Universidade XYZ</code>
<code>CN</code>	Common Name (Nome)	<code>Aluna de Teste</code>

Neste exercício o certificado é **autoassinado**: Subject e Issuer são a mesma entidade.

16. Codificação: ASN.1, DER, Base64 e PEM

Um certificado X.509 é descrito por uma gramática abstrata (**ASN.1**) e serializado em bytes:

Camada	O que é
ASN.1	A "linguagem" que define campos e tipos (SEQUENCE, INTEGER, OID...)
DER	Distinguished Encoding Rules — codificação binária canônica (1 forma válida)
BER	Variante mais permissiva do DER
Base64	Converte os bytes DER em texto ASCII
PEM	O Base64 entre marcadores <code>-----BEGIN-----</code> / <code>-----END-----</code>

Cada campo carrega um **OID** (Object Identifier). Ex.: 2.5.4.3 = Common Name; 1.2.840.113549.1.1.11 = sha256WithRSAEncryption.

Estrutura ASN.1 (simplificada):

```
Certificate ::= SEQUENCE {
    tbsCertificate      TBSCertificate,      -- "to be signed": tudo o que é assinado
    signatureAlgorithm  AlgorithmIdentifier, -- ex.: sha256WithRSAEncryption
    signatureValue      BIT STRING          -- a assinatura propriamente dita
}
TBSCertificate ::= SEQUENCE {
    version, serialNumber, signature,
    issuer, validity, subject,
    subjectPublicKeyInfo,              -- chave pública do titular
    extensions [3] EXPLICIT Extensions OPTIONAL
}
```

A AC assina os bytes DER de `tbsCertificate` e coloca o resultado em `signatureValue` — por isso qualquer alteração invalida a assinatura.

17. Extensões X.509 v3

A versão 3 adicionou **extensões**, marcadas como **críticas** (rejeitar se não entendidas) ou **não críticas**:

Extensão	Função
BasicConstraints	Indica se é AC (<code>CA=true</code>) e o comprimento máximo da cadeia
KeyUsage	Usos da chave (assinatura, não-repúdio, cifragem de chave, <code>keyCertSign...</code>)
ExtendedKeyUsage (EKU)	Usos específicos (TLS servidor/cliente, assinatura de código, e-mail...)
SubjectAlternativeName (SAN)	Nomes adicionais (DNS, IP, e-mail); em TLS é o que vale para o domínio
SubjectKeyIdentifier (SKI)	Identificador da chave pública deste certificado
AuthorityKeyIdentifier (AKI)	Aponta para a chave da AC emissora (liga o certificado ao emissor)
CRLDistributionPoints	URLs da lista de revogação (CRL)
AuthorityInfoAccess (AIA)	URLs do respondedor OCSP e do certificado da AC

Esta ferramenta inclui `BasicConstraints (CA=false)`, `KeyUsage` (assinatura digital + não-repúdio + cifragem de chave) e `SubjectAlternativeName` (e-mail).

18. Validação da cadeia de confiança

Um certificado é validado montando uma **cadeia** até uma **âncora de confiança** (root CA) no *trust store* do sistema/navegador:

```
Certificado folha (titular) → AC intermediária → AC Raiz (autoassinada, confiável)
```

Para cada elo (RFC 5280 — *path validation*), verifica-se:

1. A **assinatura** confere com a chave pública do emissor (elo acima)?
2. Está **dentro da validade** (`notBefore/notAfter`)?
3. O emissor podia emitir (`BasicConstraints CA=true` e `keyCertSign`)?
4. O nome/SAN corresponde ao esperado (ex.: o domínio acessado)?
5. **Não foi revogado** (CRL/OCSP)?
6. A cadeia termina em uma **raiz confiável**?

Como o certificado desta atividade é autoassinado, sua cadeia tem 1 elo e não termina em raiz confiável conhecida — por isso navegadores exibiriam aviso de "não confiável". Isso é esperado.

19. Revogação: CRL e OCSP

Um certificado pode precisar ser **revogado** antes do vencimento (chave comprometida, dados incorretos, desligamento):

Mecanismo	Como funciona	Trade-off
CRL	A AC publica lista assinada com os seriais revogados	Lista grande / pode estar desatualizada
OCSP	Consulta em tempo real o status de um certificado	Latência e privacidade
OCSP Stapling	O servidor TLS anexa uma resposta OCSP recente no handshake	Resolve latência e privacidade

20. TLS/HTTPS — onde os certificados são usados

O cadeado do navegador é o uso mais comum de certificados X.509. **Handshake TLS** (simplificado):

1. Cliente envia algoritmos suportados (*ClientHello*).
2. Servidor responde e **envia seu certificado** (e a cadeia).
3. Cliente **valida a cadeia** e confere o domínio no SAN.
4. Negociam uma **chave de sessão simétrica** (hoje via ECDHE, com *forward secrecy*).
5. Todo o tráfego passa a ser cifrado com algoritmo **simétrico** rápido (ex.: AES-GCM).

Ou seja: o RSA/ECC do certificado **autentica** o servidor e protege a troca da chave; o conteúdo trafega com criptografia simétrica.

21. Criptografia de curvas elípticas (ECC)

A **ECC** oferece segurança equivalente ao RSA com **chaves muito menores** (baseada no problema do logaritmo discreto em curvas elípticas):

Segurança equivalente	RSA	ECC
~112 bits	2048 bits	224 bits
~128 bits	3072 bits	256 bits
~256 bits	15360 bits	521 bits

Algoritmos: **ECDSA** (assinar), **ECDH** (troca de chaves); curvas como P-256 e Curve25519. Preferida em dispositivos móveis e IoT. A ICP-Brasil também admite certificados baseados em ECC.

22. Ameaça quântica e criptografia pós-quântica

O **algoritmo de Shor**, em um computador quântico suficientemente grande, fatoraria n (e resolveria o log discreto) em tempo polinomial — **quebrando RSA e ECC**. Esses computadores ainda não existem em escala prática, mas motivam a **criptografia pós-quântica (PQC)**.

Em 2024 o NIST padronizou os primeiros algoritmos resistentes, baseados em **reticulados (lattices)**: **ML-KEM** (Kyber, troca de chaves) e **ML-DSA** (Dilithium, assinaturas). A migração das PKIs para PQC é um dos grandes temas atuais de segurança.

23. A família de padrões PKCS

Os **PKCS** (Public-Key Cryptography Standards) padronizam formatos e operações:

Padrão	Para quê
PKCS#1	Operações e padding do RSA (v1.5, OAEP, PSS)
PKCS#7 / CMS	Mensagens assinadas/cifradas (base do S/MIME e do CAdES)
PKCS#8	Formato de chave privada (usado no <code>private_key.pem</code> desta ferramenta)
PKCS#10	CSR — Certificate Signing Request (pedido de emissão à AC)
PKCS#11	API para tokens/HSM (hardware criptográfico — A3/A4)
PKCS#12 (.pfx/.p12)	Agrupar certificado + chave privada em um único arquivo protegido

Numa emissão real, geraríamos a chave, criaríamos um **CSR (PKCS#10)** e o enviaríamos à AC, que devolveria o certificado assinado. Nesta simulação, como somos a própria AC, pulamos o CSR.

24. ICP-Brasil em profundidade

A **ICP-Brasil** (Medida Provisória nº **2.200-2/2001**) é a hierarquia oficial que confere **validade jurídica** aos certificados no país. O art. 10 estabelece a **presunção de veracidade** das assinaturas feitas com certificados ICP-Brasil.

Cadeia de confiança:

ITI — AC Raiz → Autoridade Certificadora (AC) → Autoridade de Registro (AR) → Titular

- **ITI (AC Raiz):** Instituto Nacional de Tecnologia da Informação; topo da cadeia, credencia as demais.
- **AC:** emite os certificados seguindo as **Políticas de Certificado (PC)** e a **Declaração de Práticas de Certificação (DPC)**.
- **AR:** faz a **validação presencial ou por videoconferência** da identidade do solicitante.

Séries (requisito de segurança × armazenamento):

Série	Geração da chave	Armazenamento	Chave mínima
A1 / S1	Software	Arquivo (HD)	RSA 2048
A2 / S2	Software	Cartão/token sem hw cripto	RSA 2048
A3 / S3	Hardware	Token/smartcard cripto	RSA 2048
A4 / S4	Hardware	HSM	RSA 4096

A letra **A** (Assinatura) é para assinatura/autenticação; **S** (Sigilo) para cifragem/confidencialidade. A chave de hardware (A3/A4) **nunca sai** do dispositivo (via PKCS#11).

Certificados de pessoa e aplicações: e-CPF (pessoa física), e-CNPJ (pessoa jurídica), NF-e/CT-e (documentos fiscais), SSL/TLS ICP-Brasil (sites).

Padrões de assinatura (perfis AdES — ETSI; DOC-ICP-15):

Padrão	Base	Uso típico
CAdES	CMS / PKCS#7	Arquivos binários (.p7s)
XAdES	XML	NF-e e documentos XML
PAdES	PDF	Assinatura embutida em PDF

Esses perfis preveem **carimbo do tempo** (ACT) e inclusão de informações de revogação, permitindo a verificação **no longo prazo** (LTV — Long-Term Validation), mesmo após o certificado expirar.

Lei 14.063/2020 define ainda as assinaturas eletrônicas no setor público: simples, avançada e qualificada (esta última baseada em certificado ICP-Brasil).

25. Boas práticas de segurança

- ☐ Proteja a **chave privada** com senha forte e nunca a compartilhe.
 - ☐ Mantenha o arquivo da chave com **permissões restritas** de acesso.
 - ☒ Em ambientes reais, chaves comprometidas devem ser **revogadas** junto à AC (CRL/OCSP).
 - ☐ Acompanhe a **validade** e renove antes do vencimento.
 - ☐ Prefira armazenar a chave em **hardware seguro** (token/smartcard, A3/A4) quando possível.
 - ☐ Reforço: este exercício é **simulado** e não envolve uma AC credenciada.
-

26. Referências

- **RFC 5280** — Internet X.509 PKI Certificate and CRL Profile.
 - **RFC 8017** — PKCS#1 v2.2: RSA Cryptography Specifications (OAEP, PSS).
 - **RFC 6960** — OCSP (Online Certificate Status Protocol).
 - **RFC 8446** — TLS 1.3.
 - **RFC 7468** — Textual encodings of PKIX (formato PEM).
 - **RFC 5652** — CMS (Cryptographic Message Syntax), base do CAdES/S-MIME.
 - **ITU-T X.509** — recomendação que define o certificado.
 - **ITU-T X.690** — regras de codificação ASN.1 (BER/CER/DER).
 - **FIPS 180-4** — Secure Hash Standard (SHA-2).
 - **FIPS 186-5** — Digital Signature Standard (RSA, ECDSA, EdDSA).
 - **FIPS 203/204** — ML-KEM e ML-DSA (criptografia pós-quântica, 2024).
 - **MP 2.200-2/2001** — institui a ICP-Brasil.
 - **Lei 14.063/2020** — assinaturas eletrônicas no setor público.
 - **DOC-ICP-04 / DOC-ICP-15** — DPC e padrões de assinatura da ICP-Brasil.
 - Documentação da biblioteca **cryptography** (pyca) — <https://cryptography.io>.
-

27. Glossário

Termo	Significado
RSA	Criptografia assimétrica baseada na fatoração de inteiros
ECC / ECDSA	Criptografia de curvas elípticas; ECDSA é a assinatura correspondente
AES	Algoritmo simétrico padrão para cifrar dados em volume
X.509	Padrão de estrutura de certificados digitais
DN	Distinguished Name — atributos que identificam o titular
CN	Common Name — nome comum do titular
SAN	Subject Alternative Name — nomes alternativos (DNS, IP, e-mail)
OID	Object Identifier — identificador numérico global de campo/algoritmo
ASN.1	Notação abstrata que descreve a estrutura do certificado
DER / BER	Regras de codificação binária do ASN.1 (DER é canônico)
PEM	Bytes DER em Base64, com marcadores BEGIN/END
PKCS#1/#8/#10/#11/#12	Padrões de RSA, chave privada, CSR, tokens/HSM e bundle cert+chave
Hash / SHA-256	Resumo digital de tamanho fixo usado em assinaturas
Fingerprint	Hash do certificado inteiro, usado para identificá-lo
Padding (OAEP/PSS)	Preenchimento que torna o RSA seguro (probabilístico)
PKI / ICP	Public Key Infrastructure — infraestrutura de chaves públicas
AC / CA	Autoridade Certificadora — emite certificados
AR / RA	Autoridade de Registro — valida a identidade do solicitante
ACT	Autoridade de Carimbo do Tempo — emite timestamps confiáveis
ITI	Instituto Nacional de Tecnologia da Informação — AC Raiz da ICP-Brasil
Trust store	Conjunto de raízes confiáveis instaladas no sistema/navegador
CRL / OCSP	Mecanismos para verificar se um certificado foi revogado
Forward secrecy	Comprometer a chave de longo prazo não revela sessões passadas
AdES (CAAdES/XAdES/PAAdES)	Padrões de assinatura avançada (arquivos, XML, PDF)
PQC	Criptografia pós-quântica (ex.: ML-KEM, ML-DSA)
Autoassinado	Certificado em que titular e emissor são a mesma entidade

Documento elaborado para a atividade educacional de Geração de Certificados Digitais Simulados — ICP-Brasil. Para emissões reais, procure uma Autoridade Certificadora credenciada na ICP-Brasil.